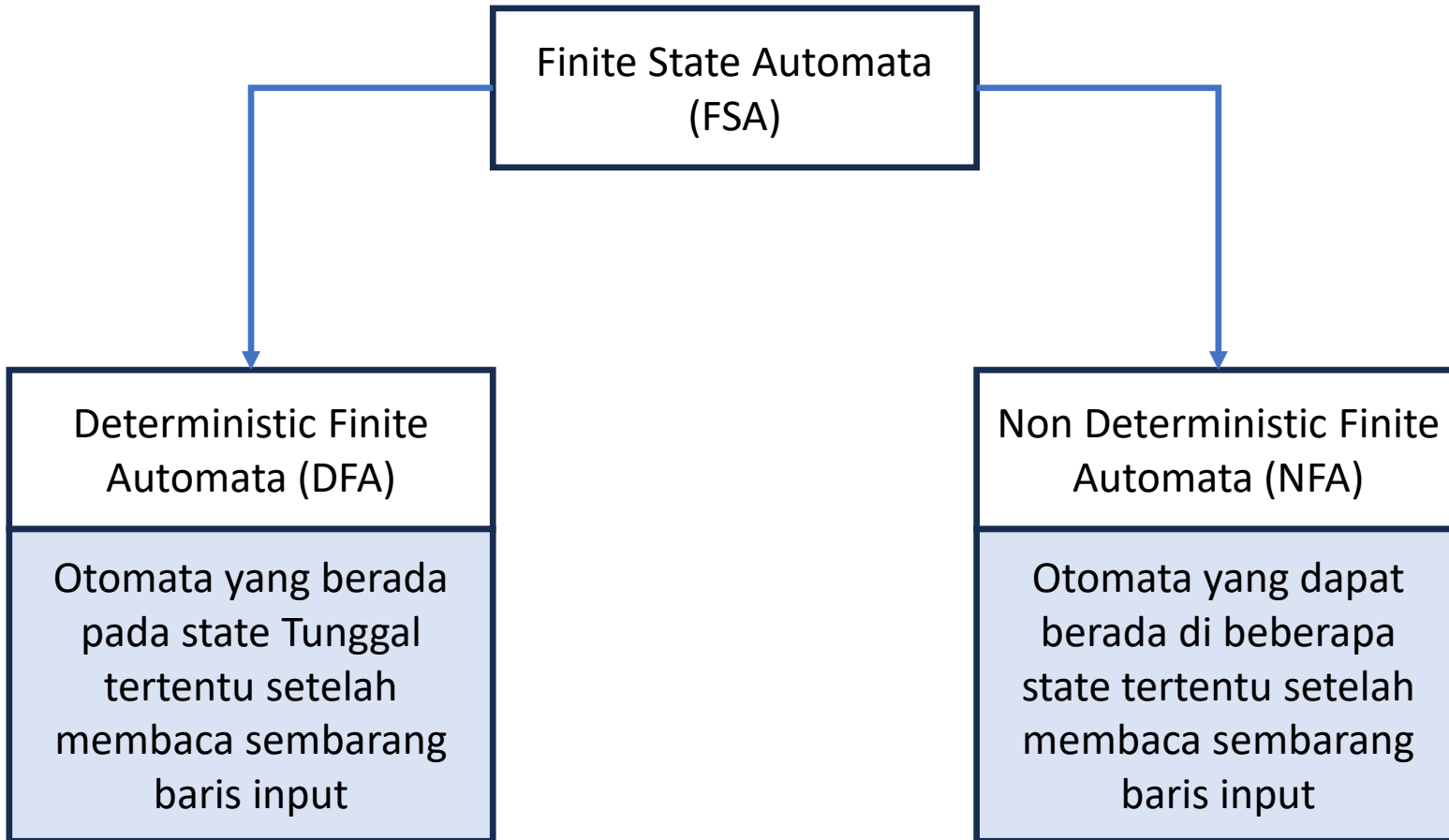


Pert. 4

Deterministik Finite Automata

TEORI BAHASA DAN OTOMATA



Pengantar DFA

- ❑ DFA (Deterministic Finite Automaton) adalah model mesin abstrak yang digunakan untuk mengenali *regular language*.
- ❑ DFA memiliki satu keadaan awal, sejumlah keadaan hingga, dan satu atau lebih keadaan akhir (*accepting states*).
- ❑ Transisi dari satu keadaan ke keadaan lainnya tergantung pada input simbol, dan untuk setiap keadaan serta input simbol, hanya ada satu transisi yang mungkin (*deterministik*).

Deterministic Finite Automata (DFA)

DFA dapat didefinisikan sbb :

$$M = (Q, \Sigma, \delta, S, F)$$

- Q = a non-empty finite set
- Σ = input alphabet
- S = initial start state
- F = set of final states
- δ = fungsi transisi
- DFA dapat digunakan sebagai finite accetor/language recognizer : menerima suatu string tertentu tapi menolak yang lainnya

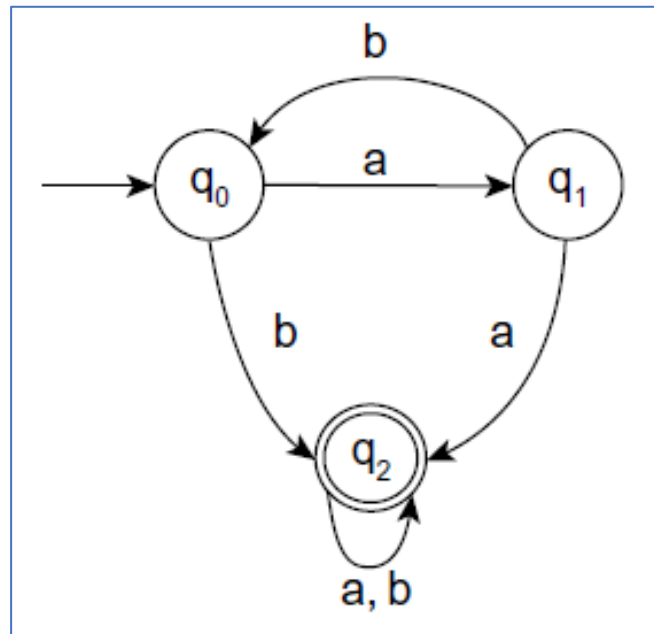
Notasi DFA

$$M = \{ \{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\} \}$$

1. Fungsi Transisi

$$\begin{array}{ll} \delta(q_0, a) = q_1 & \delta(q_0, b) = q_2 \\ \delta(q_1, a) = q_2 & \delta(q_1, b) = q_0 \\ \delta(q_2, a) = q_2 & \delta(q_2, b) = q_2 \end{array}$$

2. Diagram Transisi

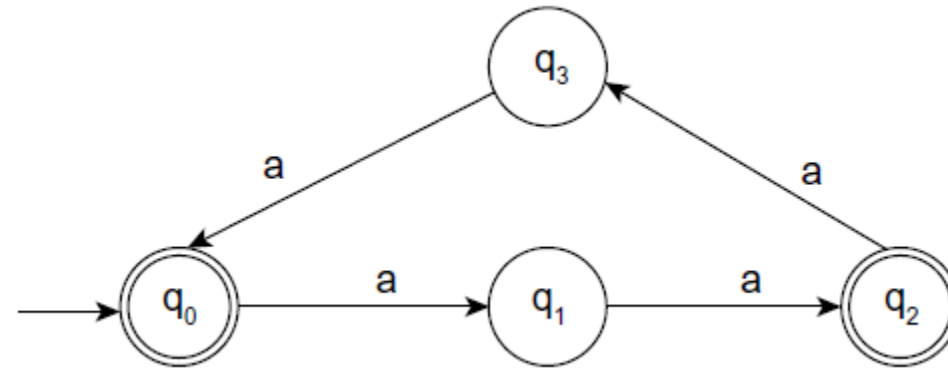
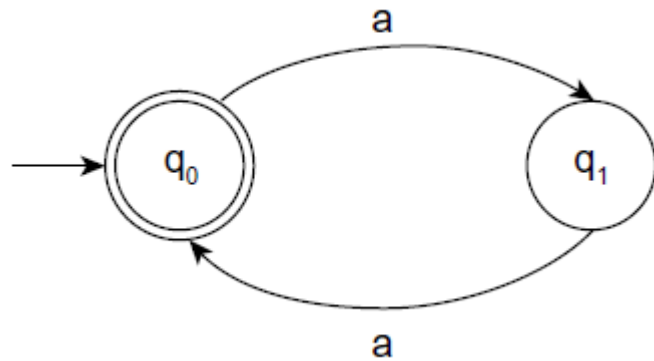


3. Tabel Transisi

Δ/Σ	a	b
$\rightarrow q_0$	q_1	q_2
q_1	q_2	q_0
$*q_2$	q_2	q_2

Contoh

Buat gambar DFA yang menerima input karakter “a” dengan jumlah genap



- Kedua DFA dapat menerima Bahasa yang sama tapi beda jumlah state
- Jumlah state yang lebih sedikit lebih baik karena implementasi lebih sederhana

Buatkan table transisi dan fungsi transisinya!

Minimisasi/penyederhanaan DFA

- Penyederhanaan DFA: Proses mengurangi jumlah state tanpa mengubah bahasa yang dikenali oleh DFA tersebut
- DFA yang sederhana : memiliki kompleksitas dan kinerja yang lebih baik
- Setiap pasangan state dalam FSA dikelompokan:
 - Distinguishable : pasangan state yang dapat dibedakan
 - Indistinguishable : pasangan state yang tidak dapat dibedakan

Distinguishable-indistinguishable

- Dua buah state p dan q dari sebuah FSA dikatakan **distinguishable** jika ada string $w \in \Sigma^*$ sedemikian sehingga :
 - $\delta(q, w) \in F$
 - Sedangkan $\delta(p, w) \notin F$
- Dua buah state p dan q dari sebuah FSA dikatakan **indistinguishable** jika untuk semua $w \in \Sigma^*$:
 - $\delta(q, w) \in F$ begitu pula $\delta(p, w) \in F$
 - $\delta(q, w) \notin F$ begitu pula $\delta(p, w) \notin F$

Indistinguishable States (State yang Tidak Dapat Dikenali Perbedaannya)

❑ **Definisi:** Dua buah state dikatakan **indistinguishable** (tidak dapat dibedakan) jika:

Untuk *semua string input* yang mungkin, kedua state tersebut akan:

- Berakhir pada **accepting state** atau
- Berakhir pada **non-accepting state** secara **bersamaan**.

❑ **Artinya:**

Jika tidak bisa menemukan *string apapun* yang bisa membuat salah satu state menerima dan lainnya menolak, maka kedua state itu **ekivalen** — bisa digabung menjadi satu saat proses minimisasi.

Contoh: Jika q_1 dan q_2 selalu merespons semua string dengan cara yang sama (misal: menerima semua string yang sama), maka $q_1 \equiv q_2 \rightarrow \text{indistinguishable}$.

Distinguishable States (State yang Dapat Dikenali Perbedaannya)

❑ **Definisi:** Dua buah state dikatakan **distinguishable** (dapat dibedakan) jika:

- Ada **setidaknya satu string input** yang membuat kedua state memberikan hasil yang **berbeda**:
- Satu menerima (accepting), dan satu lagi menolak (non-accepting).

❑ **Artinya:**

Kedua state tidak bisa digabung dalam minimisasi karena memiliki perilaku yang berbeda terhadap input tertentu.

❑ **Contoh:** Jika:

- Dari q3, string “10” berakhir di state yang menerima
- Dari q4, string “10” berakhir di state yang menolak

Maka q3 dan q4 $\rightarrow$ *distinguishable*.

Peran Penting Dalam Minimisasi DFA

- Menggabungkan semua **indistinguishable** state → menjadi satu state.
- Memastikan semua **distinguishable** state → tetap dipisah.

Ini dilakukan menggunakan **tabel pasangan (state pairing table)** atau metode **partisi ekivalensi**.

Relasi

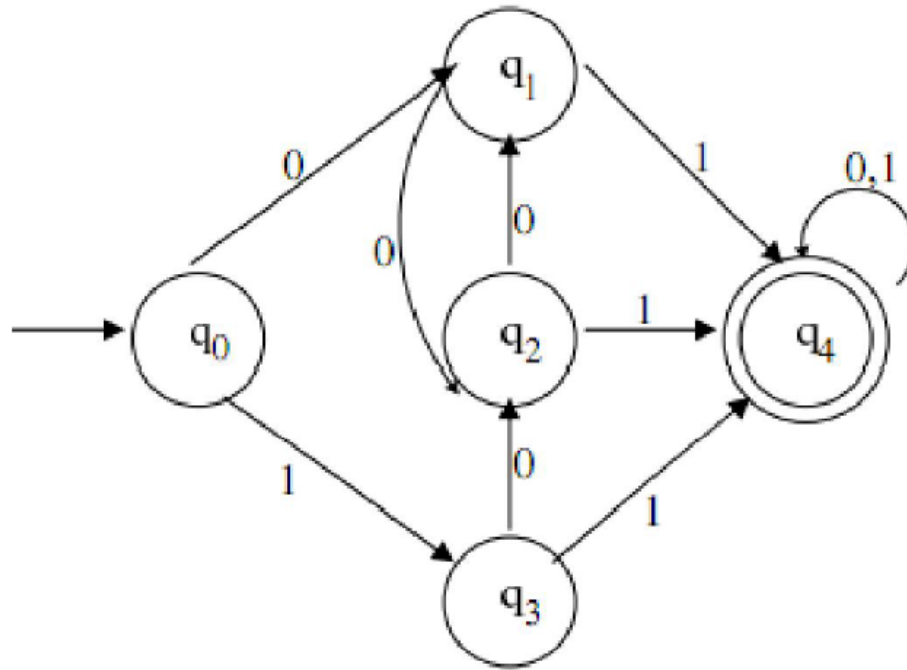
- Pasangan dua buah state memiliki salah satu kemungkinan :
 - Distinguishable atau
 - Indistinguishable
 - Contoh :
 - Jika p dan q indistinguishable
 - q dan r juga indistinguishable p, q, r indistinguishable
 - Jika p dan q distinguishable
 - q dan r juga distinguishable
- 
- p, q, r distinguishable

Langkah penyederhanaan

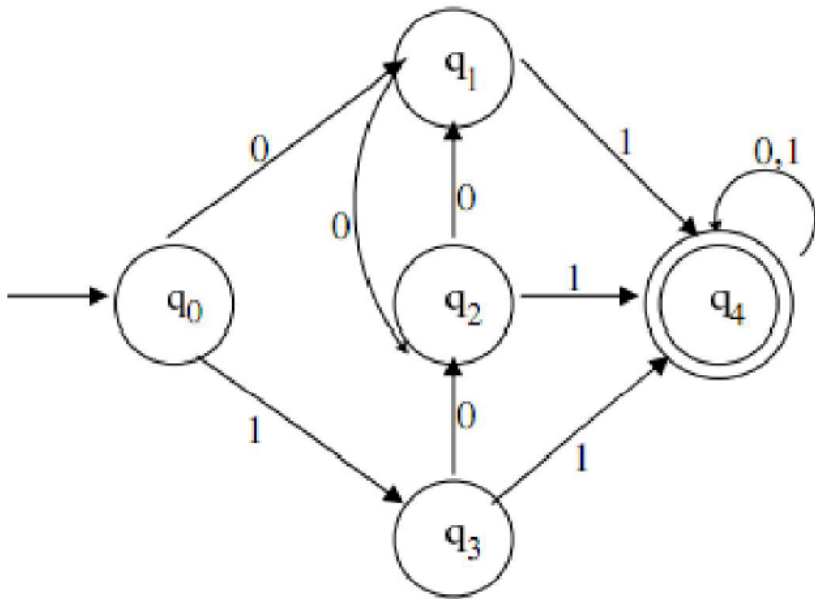
1. Hapus semua useless state
2. Buatlah semua pasangan state (p,q) yang distinguishable dimana $p \in F$ dan $q \notin F$. Catat semua pasangan state tersebut
3. Untuk semua state, lakukan pencarian state lainnya yang distinguishable dengan urutan :
 - Untuk semua (p,q) dan semua $a \in \Sigma$
 - Hitung $\delta(p,a) = p_a$ dan $\delta(q,a) = q_a$
 - Jika pasangan (p_a, q_a) merupakan pasangan state yang distinguishable
 - Maka pasangan (p, q) juga merupakan pasangan state yang distinguishable
4. Semua pasangan state yang tidak termasuk distinguishable maka state tersebut adalah indistinguishable
5. Beberapa state yang indistinguishable dapat digabungkan menjadi satu state
6. Sesuaikan transisi dari state-state tersebut

Contoh

Sederhanakan DFA Berikut



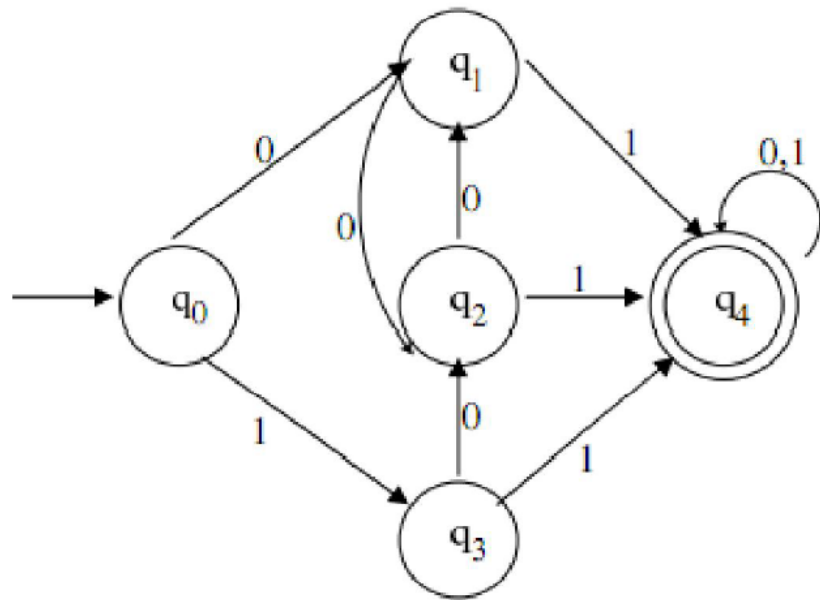
Langkah 1



Identifikasilah setiap kombinasi
state yang mungkin

(q_0, q_1)	(q_1, q_2)	(q_2, q_3)
(q_0, q_2)	(q_1, q_3)	(q_2, q_4)
(q_0, q_3)	(q_1, q_4)	(q_3, q_4)
(q_0, q_4)		

Langkah 2



State yang berpasangan dengan state akhir (q4) merupakan state yang distinguishable

(q0 , q1)

(q0 , q2)

(q0 , q3)

(q0 , q4) distinguishable

(q1 , q2)

(q1 , q3)

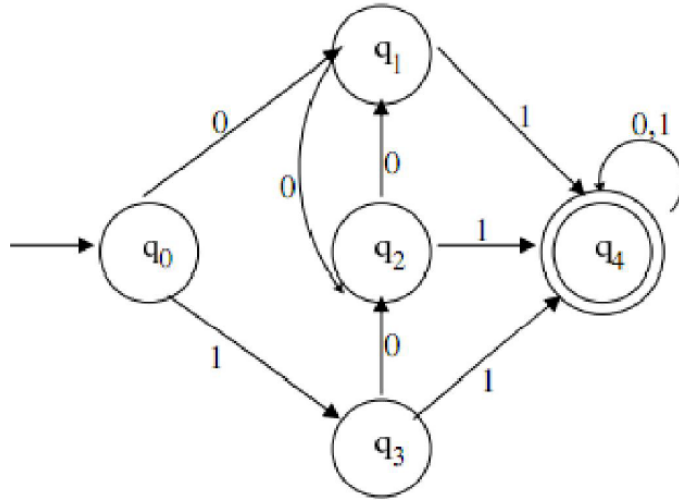
(q1 , q4) distinguishable

(q2 , q3)

(q2 , q4) distinguishable

(q3 , q4) distinguishable

Langkah 3



Untuk pasangan state yang lain jika masing-masing state mendapat input yang sama, maka bila satu state mencapai state akhir dan yang lain tidak mencapai state akhir maka dikatakan distinguishable.

Untuk (q_0, q_1) :

$$\delta(q_0, 1) = q_3$$

$$\delta(q_1, 1) = q_4$$

$$\delta(q_0, 0) = q_1$$

$$\delta(q_1, 0) = q_2$$



(q_0, q_1)

distinguishable

Untuk (q_0, q_2) :

$$\delta(q_0, 1) = q_3$$

$$\delta(q_2, 1) = q_4$$

$$\delta(q_0, 0) = q_1$$

$$\delta(q_2, 0) = q_1$$



(q_0, q_2)

distinguishable

Untuk (q_0, q_3) :

$$\delta(q_0, 1) = q_3$$

$$\delta(q_3, 1) = q_4$$

$$\delta(q_0, 0) = q_1$$

$$\delta(q_3, 0) = q_2$$



(q_0, q_3)

distinguishable

Untuk (q_1, q_2) :

$$\delta(q_1, 1) = q_4$$

$$\delta(q_2, 1) = q_4$$

$$\delta(q_1, 0) = q_2$$

$$\delta(q_2, 0) = q_1$$



(q_1, q_2)

indistinguishable

Untuk (q_1, q_3) :

$$\delta(q_1, 1) = q_4$$

$$\delta(q_3, 1) = q_4$$

$$\delta(q_1, 0) = q_2$$

$$\delta(q_3, 0) = q_2$$



(q_1, q_3)

indistinguishable

Untuk (q_2, q_3) :

$$\delta(q_2, 1) = q_4$$

$$\delta(q_3, 1) = q_4$$

$$\delta(q_2, 0) = q_1$$

$$\delta(q_3, 0) = q_2$$

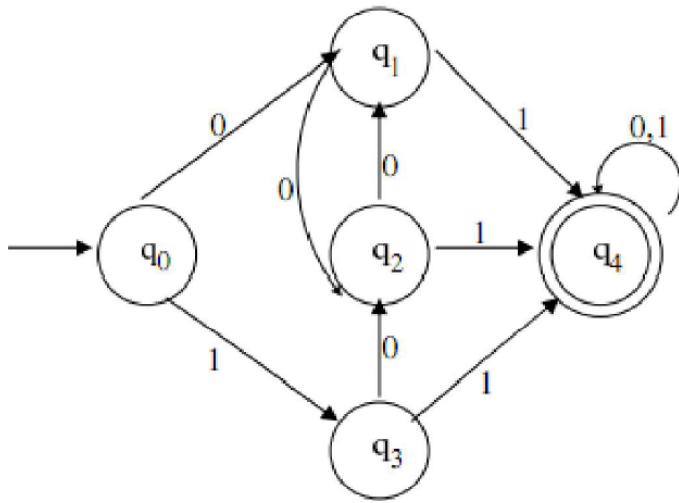


(q_2, q_3)

indistinguishable

Langkah 4

Kondisi seluruh pasangan



(q0 , q1) distinguishable

(q0 , q2) distinguishable

(q0 , q3) distinguishable

(q0 , q4) distinguishable

(q1 , q2) indistinguishable

(q1 , q3) indistinguishable

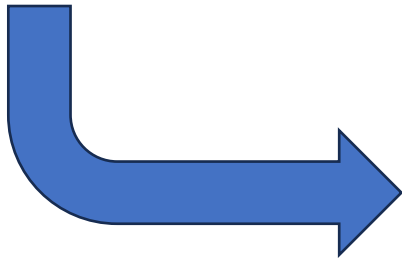
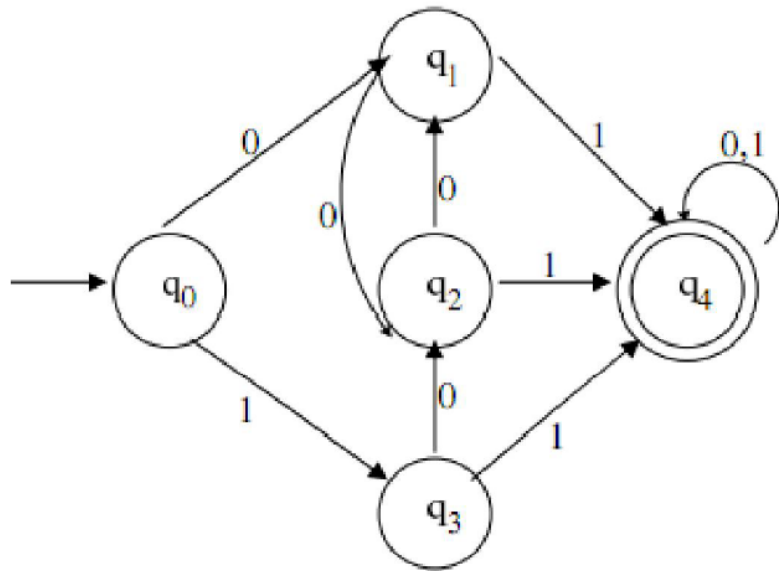
(q1 , q4) distinguishable

(q2 , q3) indistinguishable

(q2 , q4) distinguishable

(q3 , q4) distinguishable

Langkah 5



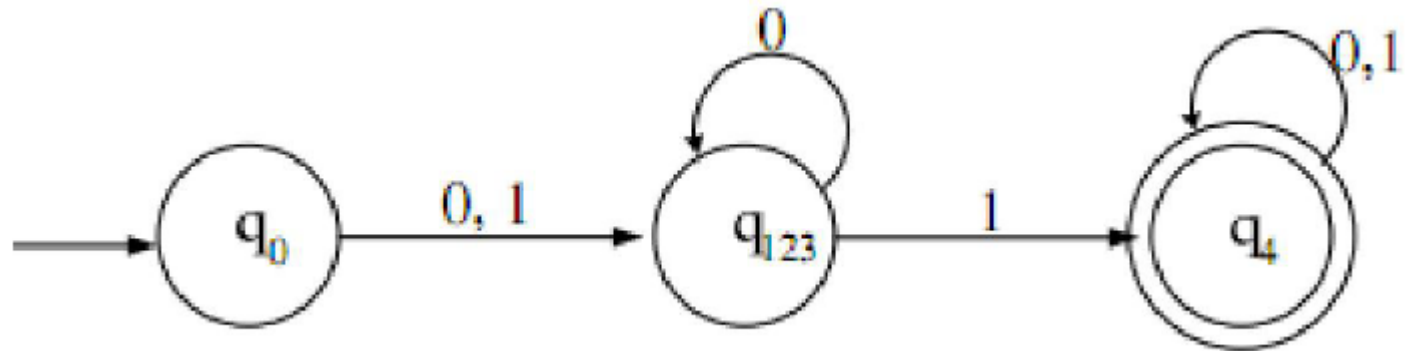
Kelompokan pasangan state yg indistinguishable

(q1 , q2) indistinguishable

(q1 , q3) indistinguishable

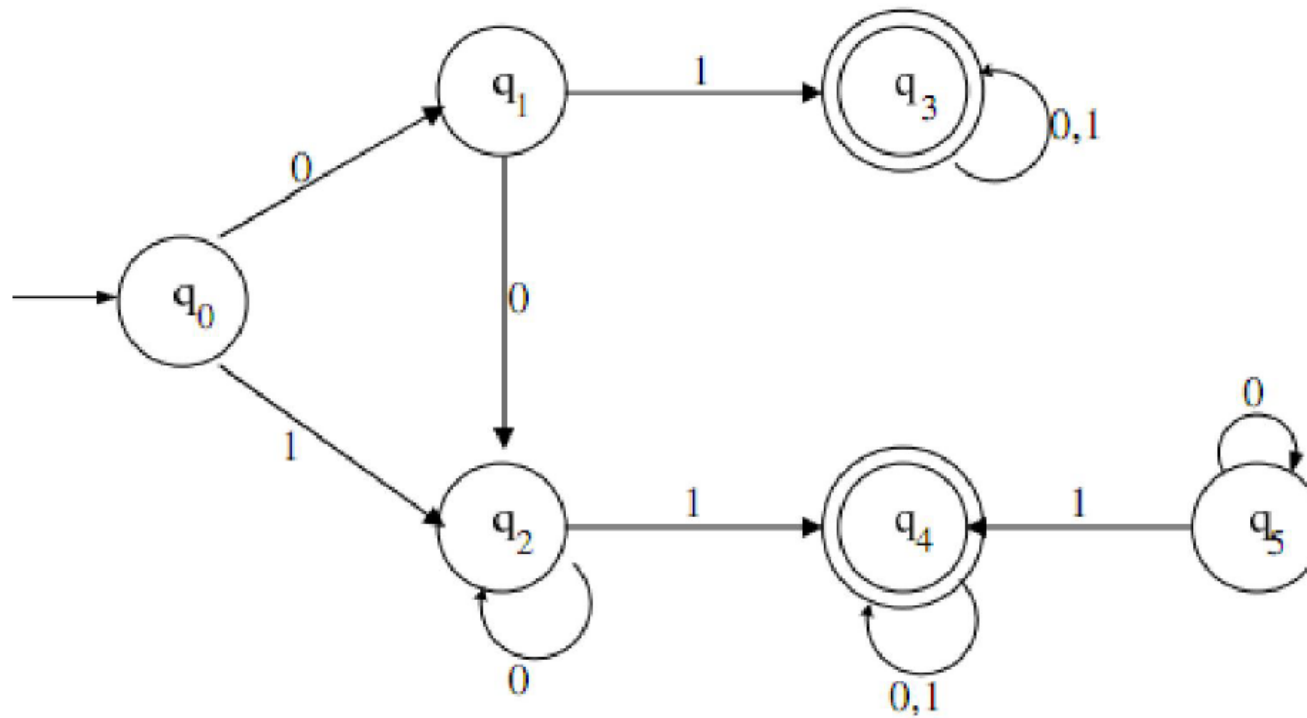
(q2 , q3) indistinguishable

Karena q1 indistinguishable dengan q2 dan q2 indistinguishable dengan q3 , maka bisa dikatakan bahwa q1 , q2 , dan q3 saling indistinguishable dan dapat dijadikan satu state.



Latihan

- Sederhanakan DFA berikut



Simulasi DFA Menggunakan Python dan JFLAP

A. Python

Contoh kode simulasi DFA sederhana:

```
dfa = {  
    'q0': {'a': 'q1', 'b': 'q0'},  
    'q1': {'a': 'q1', 'b': 'q2'},  
    'q2': {'a': 'q1', 'b': 'q0'}  
}  
start_state = 'q0'  
accept_states = {'q2'}  
def simulate_dfa(dfa, start, accept, input_string):  
    current = start  
    for symbol in input_string:  
        if symbol in dfa[current]:  
            current = dfa[current][symbol]  
        else:  
            return False  
    return current in accept  
print(simulate_dfa(dfa, start_state, accept_states, "aab")) # Output: True
```

Simulasi DFA Menggunakan Python dan JFLAP

B. JFLAP

1. Buka aplikasi JFLAP.
2. Pilih menu "Finite Automaton".
3. Tambahkan state dan transisi dengan klik kanan dan drag.
4. Tetapkan state awal dan akhir.
5. Jalankan simulasi dengan "Input → Step by State" atau "Fast Run".

Contoh Aplikasi DFA dalam Kehidupan Nyata

1. Validasi Format Input (seperti email, nomor telepon, dll)

Deskripsi:

- DFA digunakan untuk mengecek apakah input yang dimasukkan pengguna sesuai dengan aturan yang telah ditentukan.
- Contoh paling umum: validasi alamat email, password, atau format nomor.
- **Contoh Kasus:** Misalnya, untuk memvalidasi string biner yang harus **berakhir dengan “01”**, DFA dapat dirancang untuk hanya menerima string yang sesuai.

2. Autentikasi Sistem Password dengan Pola Tertentu

Deskripsi:

- DFA dapat digunakan untuk memeriksa apakah password mengikuti urutan tertentu.
- Contoh: password harus dimulai dengan huruf besar, diikuti oleh minimal satu angka, dan diakhiri simbol.

3. Sistem Menu Interaktif (mesin penjual otomatis/Vending Machine)

Deskripsi:

- DFA menggambarkan **alur navigasi menu otomatis**: setiap tombol yang ditekan pengguna membawa sistem ke state berbeda.
- Vending machine adalah representasi nyata DFA, di mana:
 - State: jumlah uang yang sudah dimasukkan
 - Input: nominal uang yang dimasukkan
 - Transisi: perubahan total uang
 - Final state: ketika uang cukup dan minuman bisa dikeluarkan

4. Spell Checker atau Autocomplete (dalam Keyboard/Editor)

Deskripsi:

- DFA bisa digunakan untuk mengenali pola kata, awalan, atau urutan huruf.
- Implementasi awal pada **auto-suggest** atau **auto-correction** keyboard berbasis urutan input.

5. Remote Control TV / Elevator Control

Deskripsi:

- Setiap tombol menimbulkan perubahan kondisi (state).
- Misal: dari channel 1 → tekan “next” → channel 2 → tekan “next” lagi → channel 3, dst.

6. Sistem elevator:

- State: posisi lantai
- Input: tombol naik/turun
- Transisi: perubahan lantai
- Final state: sampai di lantai tujuan

7. Sistem Lampu Lalu Lintas Otomatis

Deskripsi:

- DFA dapat menggambarkan **siklus lampu lalu lintas**: hijau → kuning → merah → hijau.
- Setiap lampu adalah state, dan waktu atau sensor lalu lintas menjadi input.
- Transisi dilakukan secara deterministik setelah waktu tertentu atau kondisi tertentu terpenuhi.

8. Protokol Komunikasi Digital (Parser Bitstream)

Deskripsi:

- DFA dapat memodelkan parsing bitstream dalam komunikasi digital.
- Digunakan untuk memverifikasi **format protokol**, seperti deteksi start/stop bit atau header tertentu.

9. Kompiler: Analisis Leksikal (Lexical Analyzer)

Deskripsi:

- Salah satu aplikasi terpenting DFA dalam informatika!
- Digunakan dalam tahap **lexical analysis** pada kompilasi untuk mengenali token seperti identifier, keyword, operator, dll.

Latihan 1

Desain DFA yang mengenali string yang hanya terdiri dari huruf 'a' dan 'b' dan memiliki tepat dua huruf 'b'.

Penyelesaian:

- State: q0 (0 b), q1 (1 b), q2 (2 b), q3 (lebih dari 2 b)
- Transisi:
 - q0 --b--> q1
 - q1 --b--> q2
 - q2 --b--> q3 (trap state)
 - Semua state looping untuk simbol 'a'

Latihan 2

Buat DFA yang mengenali string yang *tidak mengandung* substring "11".

Penyelesaian:

- State: q0 (awal), q1 (terakhir 1), q2 (trap state)
- Transisi:
 - q0 --0--> q0
 - q0 --1--> q1
 - q1 --0--> q0
 - q1 --1--> q2
 - q2 --0/1--> q2
- q0 dan q1 adalah *accepting states*.

Daftar Pustaka

1. Hopcroft, J.E., Motwani, R., & Ullman, J.D. (2006). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson.
2. Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.
3. Linz, P. (2016). *An Introduction to Formal Languages and Automata* (6th ed.). Jones & Bartlett Learning.
4. JFLAP Software: <http://www.jflap.org/>
5. Python Official Documentation: <https://docs.python.org/>